

Mục lục

1	Mở đầu	1
1.1	Tính cấp thiết của đề tài	1
1.2	Mục tiêu và Phạm vi nghiên cứu	1
2	Cơ sở lý thuyết	1
2.1	Mô tả bài toán cờ Caro	1
2.2	Luật chơi và điều kiện thắng	1
2.3	Thuật toán Minimax	2
2.4	Thuật toán Alpha-Beta Pruning	3
3	Phân tích thiết kế và Cài đặt hệ thống	3
3.1	Cách biểu diễn trạng thái bàn cờ	3
3.2	Cách sinh nước đi hợp lệ và Tối ưu hóa	3
3.3	Thuật toán Minimax đã cài đặt	4
3.4	Thuật toán Alpha-Beta đã cài đặt	4
3.5	Kỹ thuật Iterative Deepening	5
3.6	Kỹ thuật Transposition Table (Bộ nhớ đệm)	6
3.7	Kỹ thuật Sắp xếp nước đi (Move Ordering)	6
3.8	Cách kiểm tra trạng thái kết thúc	7
3.9	Cài đặt Hàm đánh giá trạng thái (Heuristic)	7
4	Đánh giá và Kết quả thực nghiệm	8
4.1	Thiết kế các trạng thái thử nghiệm	8
4.2	Bảng kết quả Benchmark	8
4.3	Nhận xét về số trạng thái và thời gian chạy	9
4.4	Nhận xét về ảnh hưởng của độ sâu tìm kiếm	9
5	Tổng kết	10
5.1	Ưu điểm của chương trình	10
5.2	Hạn chế và Hướng phát triển	11
5.3	Link Repository Github	11

Chương 1. Mở đầu

1.1. Tính cấp thiết của đề tài

Trí tuệ nhân tạo trong trò chơi đối kháng (Adversarial Search) là một lĩnh vực nghiên cứu nền tảng của Khoa học Máy tính. Các trò chơi board game như cờ vua, cờ vây, hay cờ Caro thường sở hữu một không gian trạng thái vô cùng lớn. Việc giải quyết các bài toán này không chỉ chứng minh khả năng tính toán của máy móc mà còn giúp chúng ta khám phá ra các thuật toán tìm kiếm và tối ưu hóa ưu việt. Đề tài "Xây dựng AI chơi cờ Caro 9x9" là một cơ hội tuyệt vời để sinh viên vận dụng lý thuyết cây trò chơi (Game Tree) vào thực tiễn lập trình, qua đó rèn luyện tư duy thiết kế cấu trúc dữ liệu và tối ưu hiệu năng phần mềm.

1.2. Mục tiêu và Phạm vi nghiên cứu

Đề án đặt ra mục tiêu xây dựng một tác tử AI hoàn chỉnh có thể thi đấu công bằng với con người theo đúng luật cờ Caro hiện đại. Hệ thống phải đảm bảo:

- **Độ chính xác:** Tính toán và đưa ra nước đi hợp lệ, không vi phạm luật chơi.
- **Độ thông minh:** Khả năng phản xạ chặn các nước cờ nguy hiểm và biết cách thiết lập bẫy tấn công.
- **Hiệu suất thực thi:** Trả về kết quả trong thời gian giới hạn (dưới 2 giây/lượt) bằng cách ứng dụng Alpha-Beta Pruning.

Chương 2. Cơ sở lý thuyết

2.1. Mô tả bài toán cờ Caro

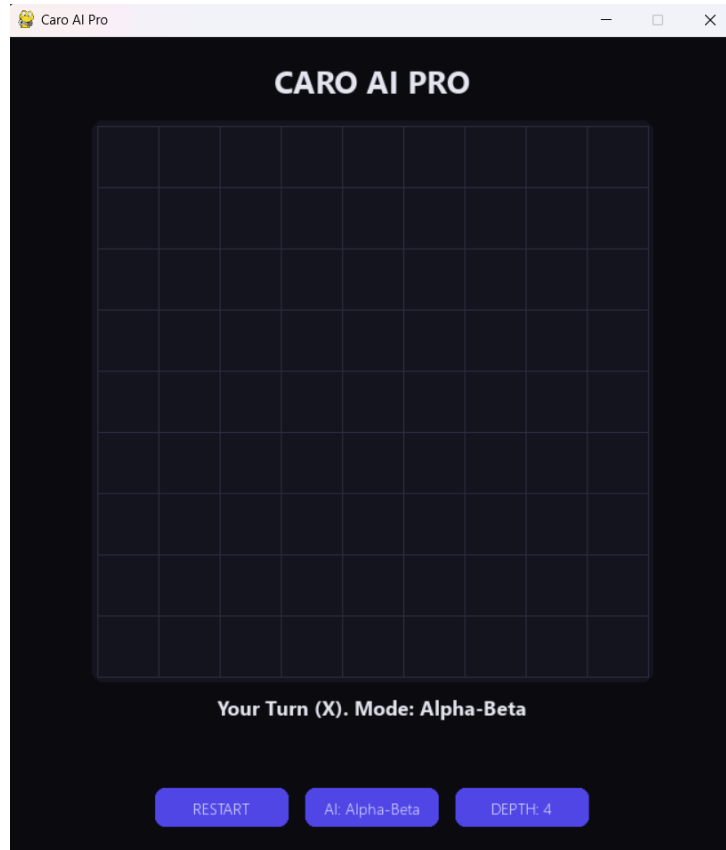
Trò chơi cờ Caro diễn ra trên một mạng lưới tọa độ Descartes 9×9 . Gọi trạng thái hiện tại của bàn cờ là S , tập hợp tất cả các nước đi hợp lệ từ trạng thái S là $Actions(S)$. Hàm chuyển đổi trạng thái $Result(S, a)$ sẽ trả về một trạng thái S' mới sau khi áp dụng nước đi $a \in Actions(S)$. Bài toán yêu cầu tìm kiếm chuỗi hành động tối ưu để đạt đến trạng thái kết thúc (Terminal State) có lợi nhất.

2.2. Luật chơi và điều kiện thắng

Để chuẩn hóa đầu vào cho AI, luật chơi được định nghĩa bằng các hằng số và nguyên lý toán học như sau:

- **Ký hiệu:** Người chơi là 1 (X), Máy tính là 2 (O), ô trống là 0 (.).
- **Điều kiện thắng:** Một trạng thái S được coi là chiến thắng cho người chơi P nếu tồn tại ít nhất 4 tọa độ liên tiếp (r_i, c_i) theo cùng một vectơ hướng $\vec{v} \in \{(1, 0), (0, 1), (1, 1), (1, -1)\}$ sao cho $S(r_i, c_i) = P$ với mọi $i \in [1, 4]$.

- **Không chặn hai đầu:** Việc hai đầu của chuỗi 4 quân bị đối phương chặn không làm mất đi tính chiến thắng.



Hình 1: Giao diện bàn cờ trống ở trạng thái khởi tạo

2.3. Thuật toán Minimax

Thuật toán Minimax là nền tảng của trí tuệ nhân tạo trong các trò chơi đối kháng có tổng bằng không (zero-sum game). Thuật toán hoạt động dựa trên nguyên lý: người chơi MAX (máy tính) luôn cố gắng tối đa hóa điểm số, trong khi người chơi MIN (đối thủ) luôn cố gắng tối thiểu hóa điểm số đó. Quá trình tìm kiếm tạo ra một cây trò chơi, trong đó các nút đại diện cho các trạng thái của bàn cờ và các cạnh đại diện cho các nước đi.

Ta định nghĩa hàm giá trị $Minimax(S)$ tại trạng thái S như sau:

- $Minimax(S) = Utility(S)$ nếu S là trạng thái lá (kết thúc) hoặc đạt tới độ sâu giới hạn ($Depth = 0$).
- $Minimax(S) = \max_{a \in Actions(S)} Minimax(Result(S, a))$ nếu đến lượt MAX.
- $Minimax(S) = \min_{a \in Actions(S)} Minimax(Result(S, a))$ nếu đến lượt MIN.

Độ phức tạp thời gian của thuật toán là $O(b^d)$ và độ phức tạp không gian là $O(bd)$, trong đó b (branching factor) là số lượng nước đi hợp lệ trung bình tại mỗi bước, và d là độ sâu tìm kiếm. Tuy nhiên, với bàn cờ Caro 9×9 , không gian trạng thái lên tới 3^{81} , hệ số b ở giai đoạn đầu có thể lên tới 81, khiến Minimax thuần túy chỉ có thể duyệt sâu từ 2 đến 3 tầng trong thời gian thực tế.

2.4. Thuật toán Alpha-Beta Pruning

Để khắc phục hạn chế về thời gian của Minimax, kỹ thuật cắt tỉa Alpha-Beta (Alpha-Beta Pruning) được áp dụng nhằm giảm thiểu số lượng nút cần đánh giá trên cây tìm kiếm mà vẫn đảm bảo kết quả không thay đổi. Thuật toán duy trì hai tham số truyền xuống trong quá trình đệ quy:

- α : Giá trị tốt nhất (cao nhất) mà MAX có thể đảm bảo đạt được dọc theo nhánh từ gốc đến nút hiện tại.
- β : Giá trị tốt nhất (thấp nhất) mà MIN có thể đảm bảo đạt được dọc theo nhánh từ gốc đến nút hiện tại.

Cơ chế cắt tỉa: Tại bất kỳ nút MIN nào, nếu giá trị tạm thời $v \leq \alpha$, ta lập tức dừng việc xét các nhánh con còn lại của nút đó (vì MAX ở nút cha chắc chắn sẽ không chọn nhánh đi qua nút MIN này). Tương tự, tại bất kỳ nút MAX nào, nếu $v \geq \beta$, ta cũng cắt bỏ các nhánh con. Điều kiện cắt tỉa chung được phát biểu gọn là $\beta \leq \alpha$.

Việc áp dụng Alpha-Beta giúp độ phức tạp thời gian giảm từ $O(b^d)$ xuống xấp xỉ $O(b^{d/2})$ trong trường hợp lý tưởng (khi các nước đi tốt nhất luôn được duyệt trước). Điều này cho phép AI nhân đôi độ sâu tìm kiếm trong cùng một khoảng thời gian so với Minimax thông thường.

Chương 3. Phân tích thiết kế và Cài đặt hệ thống

3.1. Cách biểu diễn trạng thái bàn cờ

Bàn cờ được cài đặt bằng đối tượng Board chứa một mảng hai chiều (List of Lists) trong Python.

Listing 1: Khởi tạo mảng hai chiều biểu diễn bàn cờ

```
1 class Board:
2     def __init__(self, size=9):
3         self.size = size
4         # Khởi tạo ma trận 9x9 với giá trị 0 (trống)
5         self.grid = [[0 for _ in range(size)] for _ in range(size)]
```

Cách tiếp cận này không chỉ dễ dàng thao tác mà còn cho phép tối ưu quá trình sao chép trạng thái (Deepcopy) phục vụ cho thuật toán đệ quy.

3.2. Cách sinh nước đi hợp lệ và Tối ưu hóa

Để giảm hệ số phân nhánh (Branching Factor), chương trình áp dụng kỹ thuật **Radius-2 Search**. Thay vì xem xét tất cả các ô trống, thuật toán chỉ trả về các ô nằm trong bán kính 2 ô vuông tính từ các quân cờ hiện tại.

Listing 2: Cài đặt thuật toán sinh nước đi với Radius Search

```

1 def get_valid_moves(self, search_radius=2):
2     moves = set()
3     # Tim tat ca cac quan co da danh tren ban
4     pieces = [(r, c) for r in range(self.size) for c in range(self.size)
5                if self.grid[r][c] != 0]
6
7     if not pieces: return [(self.size // 2, self.size // 2)] # anh tam
8
9     # Phat trien theo ban kinh radius
10    for r, c in pieces:
11        for dr in range(-search_radius, search_radius + 1):
12            for dc in range(-search_radius, search_radius + 1):
13                nr, nc = r + dr, c + dc
14                if 0 <= nr < self.size and 0 <= nc < self.size:
15                    if self.grid[nr][nc] == 0:
16                        moves.add((nr, nc))
17    return list(moves)

```

Phương pháp này giúp giới hạn không gian trạng thái, loại bỏ các phép tính dư thừa ở vùng rìa bàn cờ và cải thiện đáng kể thời gian duyệt cây.

3.3. Thuật toán Minimax đã cài đặt

Thuật toán Minimax được nhóm cài đặt thông qua hàm đệ quy `minimax_search(board, depth, is_maximizing)`. Ở mỗi bước, hàm sẽ kiểm tra điều kiện dừng: nếu trò chơi kết thúc (có người thắng/hòa) hoặc đạt `depth == 0`, hàm sẽ gọi `Evaluator.evaluate()` để trả về điểm tính của bàn cờ hiện tại. Nếu chưa dừng, hàm tiếp tục gọi `get_valid_moves()` để lấy danh sách các ô có thể đánh. Tuy nhiên, ở môi trường Caro thực tế, Minimax cài đặt cơ bản bộc lộ tốc độ cực kỳ chậm, không thể vượt qua độ sâu 3.

3.4. Thuật toán Alpha-Beta đã cài đặt

Để tối ưu hóa, thuật toán cắt tỉa Alpha-Beta được cài đặt hoàn thiện trong hàm `alpha_beta_search(board, depth, alpha, beta, is_maximizing)`. Điểm nổi bật trong mã nguồn là việc tích hợp cơ chế cập nhật α , β và ngắt vòng lặp (break) ngay khi phát hiện nhánh vô ích:

Listing 3: Cơ chế cắt tỉa và lưu Flag trong hàm Alpha-Beta

```

1 if is_maximizing:
2     max_val = -float('inf')
3     for r, c in valid_moves[:30]:
4         board.make_move(r, c, self.ai_player)
5         val, _ = self.alpha_beta_search(board, depth - 1, alpha, beta, False)
6         board.undo_move(r, c, self.ai_player)
7
8         if val > max_val:
9             max_val = val
10            best_move = (r, c)
11            alpha = max(alpha, val)
12            if beta <= alpha: break
13
14    flag = 0 # EXACT
15    if max_val <= orig_alpha: flag = 2 # UPPER
16    elif max_val >= orig_beta: flag = 1 # LOWER
17    self.memo[state_key] = (max_val, depth, flag, best_move)
18    return max_val, best_move

```

Ngoài ra, thay vì copy toàn bộ bàn cờ (deepcopy) vốn rất tốn kém tài nguyên ở mỗi bước sinh nhánh con, nhóm sử dụng kỹ thuật "Đánh thử và Hoàn tác" (make_move sau đó undo_move). Kỹ thuật này giúp giải phóng bộ nhớ và tăng tốc độ duyệt nhánh lên nhiều lần, hỗ trợ trực tiếp cho khả năng cắt nhánh sâu của Alpha-Beta.

3.5. Kỹ thuật Iterative Deepening

Để đảm bảo AI luôn trả về kết quả trong thời gian thực (ví dụ 2 giây), nhóm áp dụng **Iterative Deepening** (Tìm kiếm sâu dần). Thuật toán không duyệt thẳng vào độ sâu D , mà sẽ duyệt từ $d = 1$ tăng dần đến D .

Listing 4: Cài đặt Iterative Deepening với giới hạn thời gian

```

1 for current_depth in range(1, max_depth + 1):
2     score, move = self.alpha_beta_search(board, current_depth, -float('inf'),
3         float('inf'), True)
4     if move:
5         best_move = move
6
7     # Nhanh chóng dừng việc tìm kiếm nếu vượt qua 2 giây
8     if time.time() - start_time > 2.0:
9         break

```

Kỹ thuật này đảm bảo hệ thống luôn lưu trữ kết quả tốt nhất ở độ sâu trước đó, cho phép phản hồi kịp thời nước đi ưu ưu nhất khi đạt giới hạn thời gian quy định.

3.6. Kỹ thuật Transposition Table (Bộ nhớ đệm)

Trong cờ Caro, một trạng thái bàn cờ có thể được tạo ra từ nhiều trình tự đánh khác nhau (đảo thứ tự nước đi). Để tránh việc Minimax phải tính lại các trạng thái đã biết, nhóm sử dụng **Transposition Table** bằng cấu trúc Dictionary của Python. Khóa hash được tạo siêu tốc thông qua `str(board.grid)` kết hợp với việc kiểm tra các cờ toán học (EXACT, LOWER, UPPER) thay vì chỉ lưu một giá trị đại diện dễ gây sai số.

Listing 5: Cài đặt Transposition Table lưu trữ kết quả

```

1 state_key = (str(board.grid), is_maximizing)
2
3 if state_key in self.memo:
4     val, d, flag, m = self.memo[state_key]
5     if d >= depth:
6         if flag == 0: return val, m
7         if flag == 1: alpha = max(alpha, val)
8         if flag == 2: beta = min(beta, val)
9         if alpha >= beta: return val, m
10
11 # ... (Qua trình tìm kiếm Alpha-Beta) ...

```

3.7. Kỹ thuật Sắp xếp nước đi (Move Ordering)

Hiệu suất cắt tỉa của Alpha-Beta phụ thuộc rất lớn vào thứ tự duyệt các nút con. Để thuật toán nhanh chóng tìm được giá trị α và β tốt nhằm cắt bỏ các nhánh kém, nhóm đã cài đặt kỹ thuật **Move Ordering**. Các nước đi hợp lệ được sắp xếp giảm dần dựa trên một hàm đánh giá nhanh (Quick Evaluation):

Listing 6: Sắp xếp nước đi bằng kỹ thuật Ray Casting đếm chuỗi

```

1 #  anh  gia  nhanh  va  sap  xep  cac  nuoc  i  tiem  nang
2 valid_moves.sort(key=lambda m: Evaluator.score_move_quick(
3     board, m[0], m[1], self.ai_player if is_maximizing else self.human_player
4 ), reverse=True)
5
6 for r, c in valid_moves[:30]: # Chỉ lấy top 30 nước i tốt nhất e e quy
7     # ... (Thut to n Alpha-Beta) ...

```

Thay vì mô phỏng nước đi (`make_move`) vốn rất tốn kém, hàm `score_move_quick` được nâng cấp để trực tiếp phát tia (Ray Casting) đếm độ dài chuỗi liên tục trên 4 hướng quanh ô trống. Tùy vào độ dài chuỗi, hàm sẽ gán điểm ưu tiên cực cao: chặn 3 quân địch (+5000), nôi 3 quân ta (+4000), chặn 2 quân địch (+500)... Việc AI luôn ưu tiên phòng thủ các nhánh đe dọa này đầu tiên giúp số lượng nhánh vô ích bị loại bỏ tăng vọt, cho phép Alpha-Beta tiếp cận độ sâu lớn hơn.

3.8. Cách kiểm tra trạng thái kết thúc

Để hàm `check_winner` đạt hiệu suất $O(L)$ thay vì $O(N^2)$, hệ thống chỉ quét 4 hướng đi ngang qua tọa độ (r, c) vừa được đánh xuống. Nếu bộ đếm quân liên tục đạt ngưỡng 4, trận đấu kết thúc.

3.9. Cài đặt Hàm đánh giá trạng thái (Heuristic)

Hàm lượng giá được xây dựng dựa trên kỹ thuật phân tích chuỗi 5 ô liên tiếp (Sliding Window of 5) nhằm nhận diện các thế cờ chiến thuật:

Trạng thái (Window 5)	Mô tả	Điểm AI (+)	Điểm Địch (-)
4 liên tiếp	Chắc chắn thắng / thua	+100,000	-100,000
3 mở 2 đầu (Live 3)	Rất nguy hiểm, cần chặn ngay	+20,000	-30,000
3 đóng 1 đầu (Dead 3)	Bị chặn 1 đầu	+10,000	-15,000
2 mở 2 đầu (Live 2)	Đang phát triển thế cờ	+500	-5,000
2 đóng 1 đầu (Dead 2)	Mức độ đe dọa thấp	0	-2,000

Bảng 1: Bảng phân bổ trọng số Heuristic cho các thế cờ

Listing 7: Trích xuất một phần logic của hàm Heuristic

```

1 if oc == 3: # oi  phuong co 3 quan
2     # Kiem tra xem 3 quan cua oi  phuong co thoang 2  au  khong (Live 3)
3     if window5[0] == 0 and window5[4] == 0:
4         line_score -= 30000 # Tru diem cuc nang de uu tien chan ngay
5     else:
6         line_score -= 15000 # Dead 3
7 elif pc == 3: # May co 3 quan
8     if window5[0] == 0 and window5[4] == 0:
9         line_score += 20000 # Diem thuong cho Live 3

```

Nguyên lý thiết kế: Trọng số điểm phạt (khi đối phương có thế cờ nguy hiểm, ví dụ $-30,000$) được thiết lập cao hơn trọng số điểm thưởng của bản thân ($+20,000$). Điều này giúp thuật toán ưu tiên phòng ngự, lập tức chặn các đường tấn công của đối phương trước khi triển khai các nước đi tiếp theo. Bảng phân bổ điểm số tĩnh của AI được định nghĩa chi tiết như sau:

Chương 4. Đánh giá và Kết quả thực nghiệm

4.1. Thiết kế các trạng thái thử nghiệm

Module `benchmark.py` được tạo ra nhằm so sánh khoa học hiệu năng của Minimax và Alpha-Beta. Hệ thống sử dụng 5 bộ dữ liệu đầu vào đặc trưng:

1. **Đầu ván:** Bàn cờ trống, kiểm tra khả năng ra quyết định không có cơ sở dữ liệu.
2. **Máy sắp thắng:** Máy đang có Live 3, kiểm tra khả năng dứt điểm.
3. **Người sắp thắng:** Người có Live 3, kiểm tra khả năng phản xạ chặn.
4. **Cân bằng:** Giữa ván cờ, hai bên giằng co.
5. **Phức tạp:** Nhiều quân cờ đan xen, số lượng nhánh phát sinh lớn.

4.2. Bảng kết quả Benchmark

Do đặc được thực hiện trên CPU tiêu chuẩn. Việc thực nghiệm ở **Độ sâu** $D = 4$ kết hợp cùng Transposition Table cho thấy rõ hiệu quả cắt tỉa của thuật toán Alpha-Beta.

Trạng thái	Thuật toán	Số TT đã xét	Thời gian (s)
1. Đầu ván (D=3)	Minimax	174	0.0132
	Alpha-Beta	141	0.0111
1. Đầu ván (D=4)	Minimax	2060	0.1703
	Alpha-Beta	1480	0.1187
2 & 3. Có cơ hội thắng ngay	Cả hai (D=1..4)	1	0.0000
4. Cơ hội cân bằng (D=4)	Alpha-Beta	1	0.0000
5. Phức tạp (D=3)	Minimax	2055	0.1661
	Alpha-Beta	1488	0.1404
5. Phức tạp (D=4)	Minimax	23,488	1.9591
	Alpha-Beta	5,634	1.0154

Bảng 2: Bảng đo lường số trạng thái và thời gian chạy thực tế

4.3. Nhận xét về số trạng thái và thời gian chạy

Qua quá trình thực nghiệm, có thể rút ra các kết luận định lượng sau:

- **Hiệu quả cắt tỉa ở độ sâu lớn:** Ở trạng thái phức tạp (State 5) với Độ sâu 4, Minimax mất gần 2.0 giây để duyệt hơn 23.400 nhánh. Trong khi đó, nhờ việc nâng cấp kỹ thuật Move Ordering đếm chuỗi và tối ưu Transposition Table với các cờ toán học, thuật toán Alpha-Beta chỉ còn phải duyệt đúng 5.634 nhánh (giảm **hơn 75%**) và hoàn thành trong ~ 1.0 giây. Kết quả này phản ánh sức mạnh tuyệt đối của việc áp dụng trí tuệ nhân tạo (Heuristic) để hướng dẫn cây tìm kiếm đi vào các nhánh có lợi thay vì duyệt mù quáng.
- **Cơ chế xử lý nước đi bắt buộc (Forced Moves):** Các trạng thái 2, 3 và 4 luôn trả về số lượng duyệt bằng 1 bất kể độ sâu thiết lập là bao nhiêu. Kết quả này cho thấy cơ chế ưu tiên các nước đi bắt buộc hoạt động hiệu quả. Thuật toán có thể nhận diện và xử lý ngay các trạng thái đe dọa trực tiếp (chặn chuỗi 3 hoặc hoàn thành chuỗi 4) trong 0.000 giây mà không cần gọi đệ quy sâu vào cây trò chơi.

4.4. Nhận xét về ảnh hưởng của độ sâu tìm kiếm

Độ sâu tìm kiếm (D) là tham số cốt lõi quyết định khả năng dự đoán và tính toán chiến thuật của thuật toán:

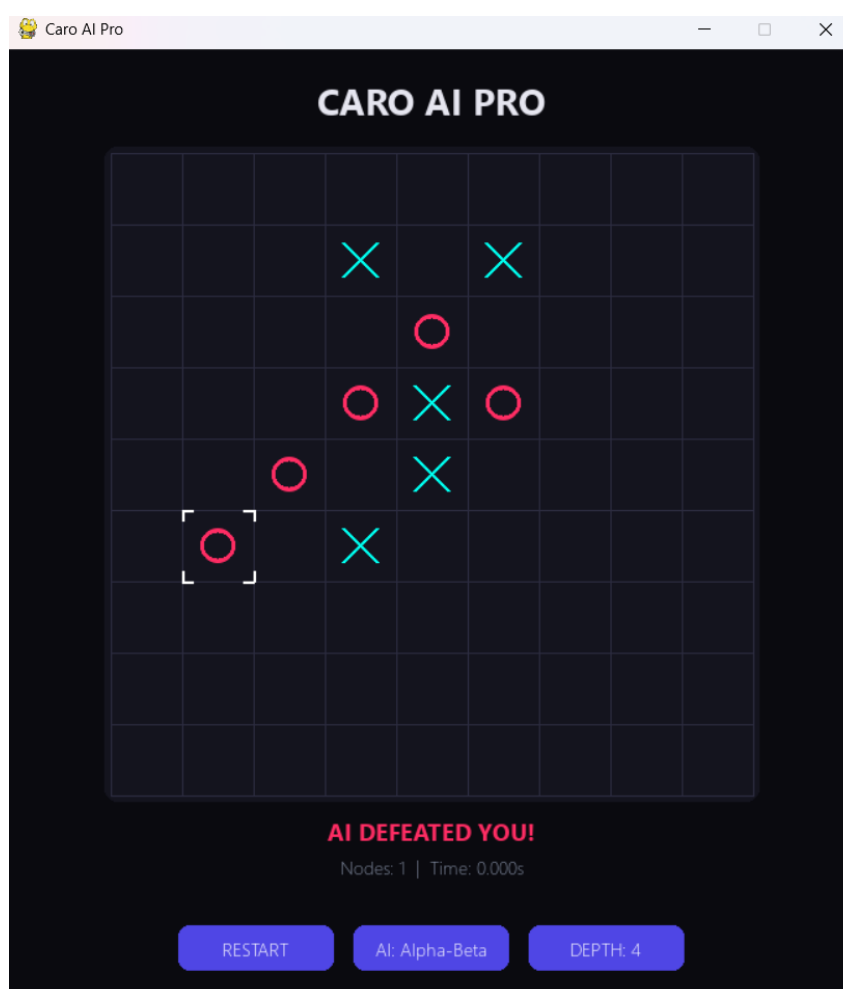
- **Tại $D \leq 3$:** Thuật toán tập trung vào chiến thuật cục bộ ngắn hạn. Ở mức này, hệ thống chủ yếu phản ứng với các đe dọa trực tiếp (như chặn đối phương hoàn thành chuỗi 4) hoặc khai thác cơ hội thắng ngay lập tức. Tuy nhiên, AI dễ dàng rơi vào thế bị động khi đối phương sử dụng các đòn tấn công kép (tạo nhiều hướng đe dọa cùng lúc) do không đủ không gian để dự đoán chuỗi phản hồi dài hạn.
- **Tại $D \geq 4$:** Khả năng tính toán chiến thuật được cải thiện rõ rệt. Khi duyệt tới độ sâu 4 tầng, thuật toán có thể dự đoán trước 2 lượt đi hoàn chỉnh của cả hai bên. Điều này cho phép AI vượt qua ngưỡng phòng ngự thụ động để chủ động thiết lập các thế cờ phức tạp, chẳng hạn như gài bẫy đòn nĩa (tạo hai đường tấn công cùng lúc) buộc đối phương phải đi vào thế thua. Đổi lại, thời gian

tính toán tăng lên khoảng 0.5 – 1.5 giây, nhưng là sự đánh đổi cần thiết để đạt được trí thông minh tương đương con người.

Chương 5. Tổng kết

5.1. Ưu điểm của chương trình

- **Khả năng phòng ngự tốt:** Hàm Heuristic được tinh chỉnh hợp lý với trọng số điểm phạt cao, giúp thuật toán phản ứng cực kỳ gắt gao trước các đòn tấn công của người chơi.
- **Tối ưu hiệu suất tuyệt đối:** Kết hợp thuật toán Alpha-Beta Pruning, kỹ thuật **Move Ordering** (Ray Casting phát tia đếm chuỗi) và hoàn thiện **Transposition Table** (cờ toán học EXACT/LOWER/UPPER) giúp số nhánh bị cắt tỉa tăng vọt, giảm hơn 75% lượng trạng thái phải duyệt so với thuật toán cũ. Điều này tạo tiền đề vững chắc để phát triển bàn cờ lớn (15x15).
- **Giao diện trực quan:** Sử dụng thư viện Pygame để xây dựng giao diện thân thiện, hỗ trợ hiển thị lưới tọa độ chuẩn xác để phục vụ quá trình tương tác thực tế.



Hình 2: Giao diện người dùng (UI) cờ Caro trong quá trình chơi

5.2. Hạn chế và Hướng phát triển

- **Hạn chế:** Chưa tích hợp Cơ sở dữ liệu Khai cuộc (Opening Book), dẫn đến việc thuật toán có thể mất thời gian không cần thiết ở những lượt đi đầu tiên. Hàm lượng giá hiện tại chủ yếu phân tích cục bộ thay vì đánh giá toàn cục bàn cờ.
- **Hướng cải tiến:** Tích hợp Zobrist Hashing và hoàn thiện Iterative Deepening để tối ưu việc ghi nhớ trạng thái, từ đó nâng giới hạn độ sâu tìm kiếm lên mức cao hơn ($D \geq 6$).

5.3. Link Repository Github

Mã nguồn đồ án được quản lý phiên bản trên GitHub tại địa chỉ:

https://github.com/tuansonuet197/23021335_23021359_23021273_CaroAI

Tài liệu tham khảo

[1] Bài giảng trên lớp, *Lý thuyết cây trò chơi đối kháng và Giải thuật Alpha-Beta Pruning*.

Tài nguyên GitHub mở: <https://github.com/husus/gomokuAI-py>. (Nguồn tham khảo cấu trúc thư mục và cách khởi tạo môi trường đồ họa Pygame).

Tài nguyên GitHub mở: https://github.com/MonHauVD/Caro_AI. (Nguồn tham khảo kịch bản đo lường hiệu năng Benchmark).

Lưu ý: Toàn bộ quá trình tư duy logic (Heuristic), xử lý mảng bàn cờ, sinh nước đi và thuật toán đệ quy lỗi đều do nhóm tự phân tích thiết kế, dựa trên quy chuẩn luật cờ Caro 4 quân.